

# **Aligarh Muslim University**

## **Department of Computer Science**

### **LAB FILE**

**Submitted By:**

Name: Sumaniya Sheikh

Enrolment Number: GR0960

Faculty Number: 25CAMSA144

Subject Code: CAMS3P01

Subject Name: Laboratory Course -III

**Submitted To:**

Dr. Faisal Anwer

Dr. Faraz Masood

Mr. Saad Shamim

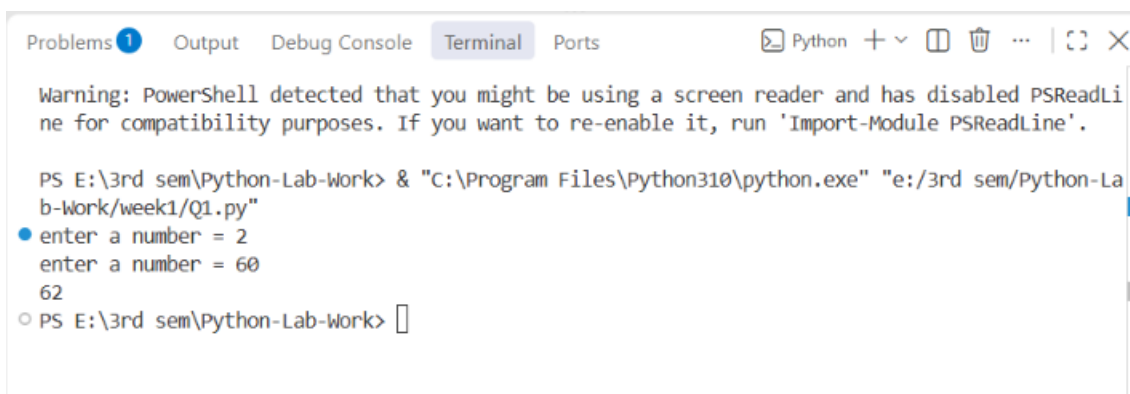
Mr. Ahsan Shameem

## Week #1

1. Write a program to find the product of two user-supplied integers and if the product is equal to or lower than 5000, then return the sum of the two numbers.

**Solution:**

```
num1,num2= int(input("enter a number = ")),int(input("enter a number = "));  
print(num1+num2 if num1*num2<=5000 else prod)
```

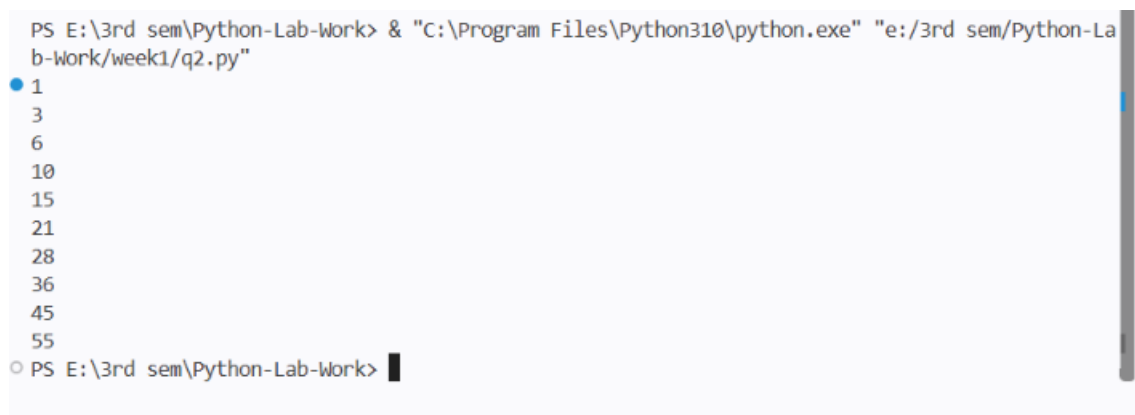


```
Problems 1 Output Debug Console Terminal Ports Python + - [ ] [X] ... [ ] [X]  
Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.  
  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week1/Q1.py"  
● enter a number = 2  
  enter a number = 60  
  62  
○ PS E:\3rd sem\Python-Lab-Work> [ ]
```

2. Write a program to print the sum of the first 10 numbers

**Solution:**

```
for i in range(1, 11):  
    sum=sum+i  
    print(sum)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week1/q2.py"  
● 1  
  3  
  6  
  10  
  15  
  21  
  28  
  36  
  45  
  55  
○ PS E:\3rd sem\Python-Lab-Work> [ ]
```

3. Write a program to iterate through a supplied list of 20 numbers and print only those numbers which are divisible by 5.

**Solution:**

```
num=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,30,24,34,32,27,35]
for i in num:
    if i%5==0:
        print(i)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q3.py"
5
10
30
35
PS E:\3rd sem\Python-Lab-Work> █
```

**4. Write a program to check if the given number is a palindrome.****Solution:**

```
num=(input("enter a number = "))
if str(num)==str(num)[::-1]:
    print("palindrome")
else:
    print("not palindrome")
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q4.py"
enter a number = 242
palindrome
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q4.py"
enter a number = 245
not palindrome
PS E:\3rd sem\Python-Lab-Work> █
```

**5. Write a program to calculate the cube of all numbers from 1 to a given number.****Solution:**

```
num=int(input("enter a three digit number = "))
for i in range(1, num):
    cube=i*i*i
    print(cube)
```

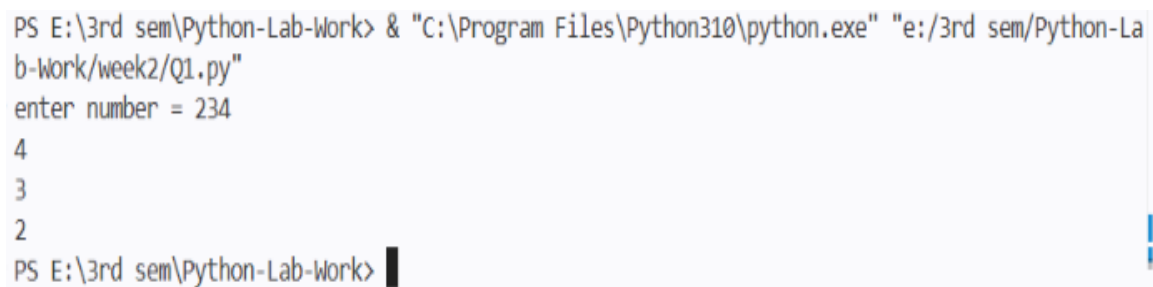
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week1/Q5.py"
enter a number = 5
1
8
27
64
125
PS E:\3rd sem\Python-Lab-Work> █
```

## Week #2

1. Write a program to extract each digit from an integer in reverse order.

### Solution:

```
num=int(input("enter number = "))
while num>0:
    digit= num%10
    print(digit)
    num=num//10
```



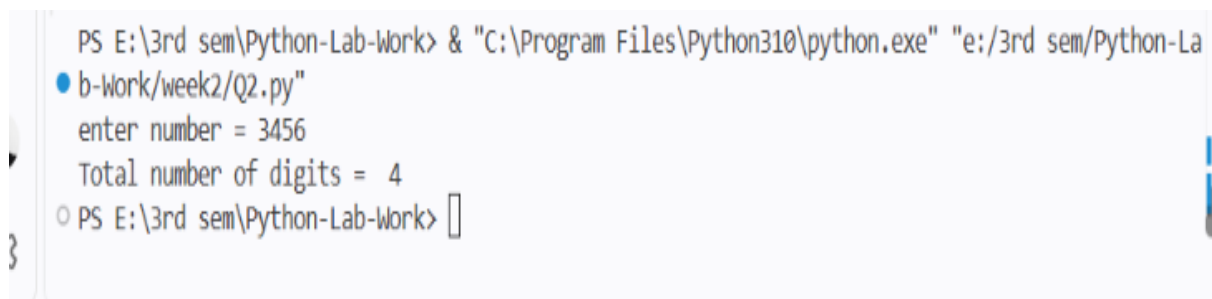
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q1.py"
enter number = 234
4
3
2
PS E:\3rd sem\Python-Lab-Work> █
```

2. Write a program to count the total number of digits in a number using a while loop.

### Solution:

```
num=int(input("enter number = "))
count=0
while num>0:
    digit= num%10
    count=count+1
    num=num//10

print("Total number of digits = ", count)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q2.py"
enter number = 3456
Total number of digits = 4
PS E:\3rd sem\Python-Lab-Work> █
```

3. Write a program to display all prime numbers within a range.

### Solution:

```
start=int(input("enter the starting number = "))
end=int(input("enter the end range = "))
for num in range(start, end+1):
    if num<2:
        continue
    is_prime =True
    for i in range(2, int(num ** 0.5)+1):
        if num%i==0:
            is_prime=False
            break
    if is_prime:
        print(num)
```

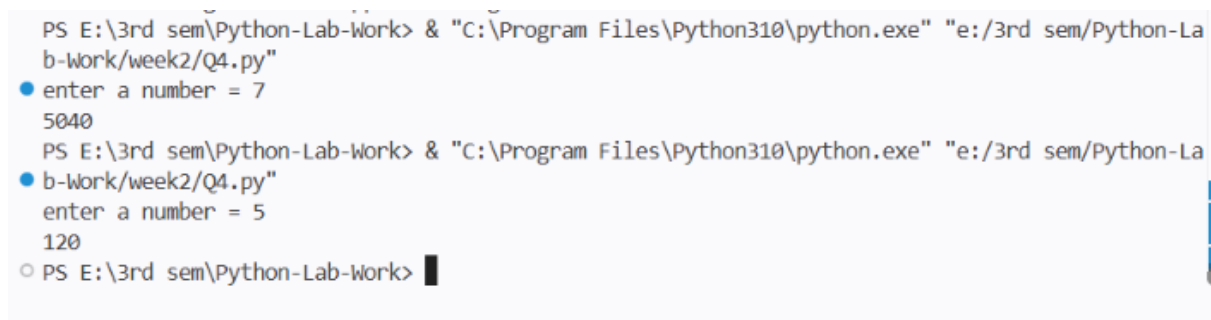


```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week2/Q3.py"
enter the starting number = 2
enter the end range = 6
prime number: 2
prime number: 3
prime number: 5
PS E:\3rd sem\Python-Lab-Work>
```

#### 4. Write a program to use a loop to find the factorial of a given number.

### Solution:

```
from functools import reduce;
n=int(input("enter a number = ")); fact=(reduce(lambda x,y: x*y,
range(1,n+1),1))
print(fact)
```

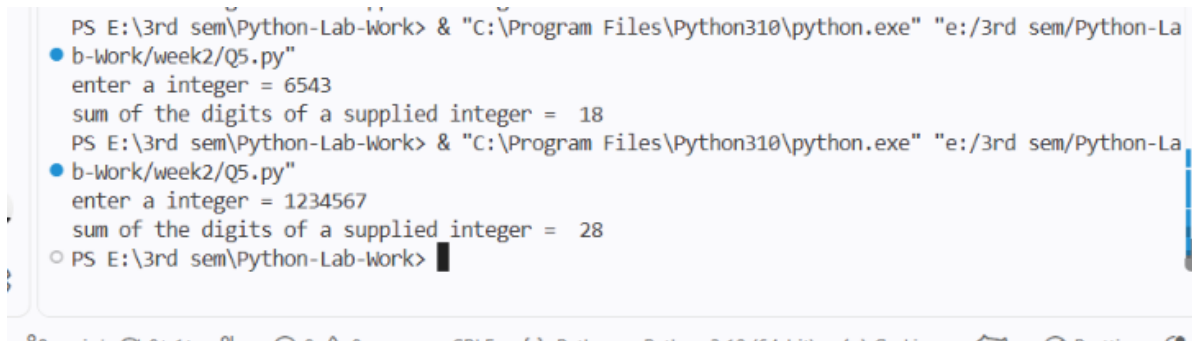


```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week2/Q4.py"
enter a number = 7
5040
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week2/Q4.py"
enter a number = 5
120
PS E:\3rd sem\Python-Lab-Work>
```

#### 5. Write a program to find the sum of the digits of a supplied integer.

**Solution:**

```
sum=sum(map(int,input("enter a integer = ")))  
print("sum of the digits of a supplied integer = ",sum)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q5.py"  
enter a integer = 6543  
sum of the digits of a supplied integer = 18  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q5.py"  
enter a integer = 1234567  
sum of the digits of a supplied integer = 28  
PS E:\3rd sem\Python-Lab-Work>
```

## Week #3

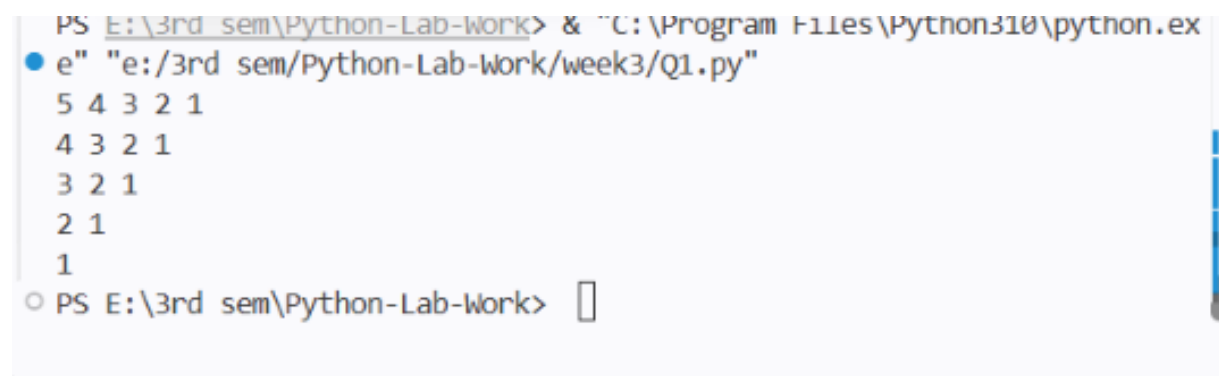
1. Write a program to print the following pattern using the for loop:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

**Solution:**

```
# for i in range(5,0,-1):
#     for j in range(i,0,-1):
#         print(j, end=" ")
#     print()
```

```
for i in range(5,0,-1):
    print(range(i,0,-1))
```



```
PS E:\3rd sem\Python-Lab-work> & "C:\Program Files\Python310\python.exe"
e" "e:/3rd sem/Python-Lab-work/week3/Q1.py"
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
PS E:\3rd sem\Python-Lab-work>
```

2. Write a program to print the following star pattern using the for loop:

```
*
**
***
****
*****
****
***
**
*
```

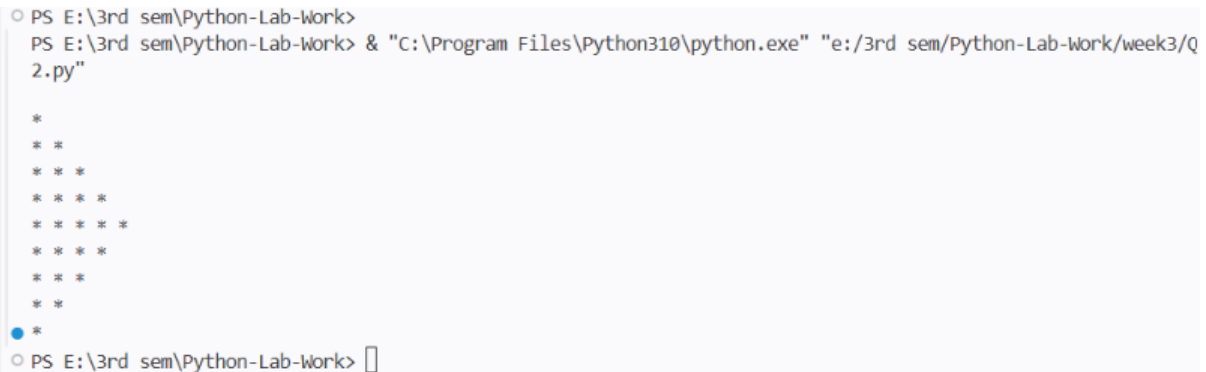
**Solution:**

```
for i in range(1, 10):
    print()
```

```

if i<=5:
    for j in range(1, i+1):
        print("*", end=" ")
else:
    for j in range(10-i):
        print("*", end=" ")

```



```

PS E:\3rd sem\Python-Lab-Work>
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q2.py"

*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*

PS E:\3rd sem\Python-Lab-Work>

```

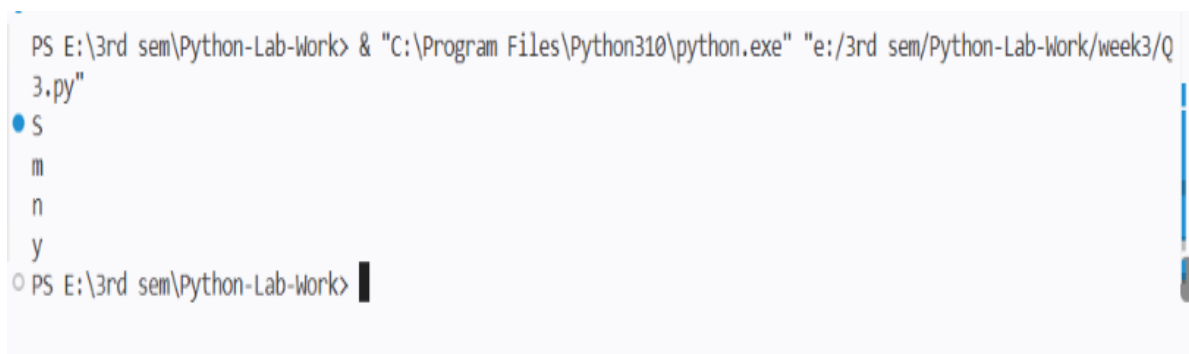
3. Write a program to print characters from a string which are present at even index numbers.

**Solution:**

```

str="Sumaniya"
for i in range(len(str)):
    if i%2==0:
        print(str[i])

```



```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q3.py"

S
u
m
a
n
i
y
a

PS E:\3rd sem\Python-Lab-Work>

```

4. Write a program to accept a string from the user and display characters that are present at even index numbers.

**Solution:**

```
str= input("enter a string = ")
for i in range(len(str)):
    if i%2==0:
        print(str[i])
```

```

• enter a string = Hello Kitty
H
l
o
K
t
y
PS E:\3rd sem\Python-Lab-Work>

```

The screenshot shows a Python IDE with a file named 5.py. The code in the file is:
 

```
str= input("enter a string = ")
for i in range(len(str)):
    if i%2==0:
        print(str[i])
```

 The output in the terminal is:
 

```

• enter a string = Hello Kitty
H
l
o
K
t
y
PS E:\3rd sem\Python-Lab-Work>

```

 The IDE status bar at the bottom shows: main\*, 0:11, Spaces: 4, UTF-8, CRLF, Python, Python 3.10 (64-bit), Go Live, Prettier.

5. Write a program to remove characters from a string starting from the nth position to the last and return a new string. Example: `remove_chars("aligarh", 3)` should output ali.

**Solution:**

```
def remove_chars(str,num):return str[:num]
str= input("enter a string = ")
num=int(input("enter the position = "))
print(remove_chars(str,num))
```

```

• enter a string = Nostalgia
enter the position = 4
Nost
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q
• 5.py"
enter a string = Meleficent
enter the position = 4
Mele
PS E:\3rd sem\Python-Lab-Work>

```

The screenshot shows a Python IDE with a file named 5.py. The code in the file is:
 

```
def remove_chars(str,num):return str[:num]
str= input("enter a string = ")
num=int(input("enter the position = "))
print(remove_chars(str,num))
```

 The output in the terminal is:
 

```

• enter a string = Nostalgia
enter the position = 4
Nost
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q
• 5.py"
enter a string = Meleficent
enter the position = 4
Mele
PS E:\3rd sem\Python-Lab-Work>

```

 The IDE status bar at the bottom shows: main\*, 0:11, Spaces: 4, UTF-8, CRLF, Python, Python 3.10 (64-bit), Go Live, Prettier.

## Week #4

1. Write a program to create a function `cal_sum_sub()` that accepts two variables and calculates addition and subtraction. Also, it must return both addition and subtraction in a single return call.

**Solution:**

```
def cal_sum_sub(a,b):  
    return a+b, a-b
```

```
a,b = map(int,input("enter first number = ").split())
```

```
print(cal_sum_sub(a,b))
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/1.py"  
enter first number = 2 4  
(6, -2)  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/1.py"  
enter first number = 4 7  
(11, -3)  
PS E:\3rd sem\Python-Lab-Work> █
```

2. Write a function to return True if the first and last number of a given list are the same. If the numbers are different, return False.

**Solution:**

```
def check_list(lst):  
    return lst[0]==lst[-1]
```

```
l= list(map(int,input("enter a list of integer = ").split()))  
print(check_list(l))
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q2.py"  
enter a list of integer = 3 4 5 6 7 5 6 7  
False  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q2.py"  
enter a list of integer = 2 3 4 5 6 2  
True  
PS E:\3rd sem\Python-Lab-Work> █
```

3. Given a list of numbers, write a program to turn every item of the list into its square.

**Solution:**

```
li= list(map(int,input("enter a list of integer = ").split()))  
print(list(map(lambda x:x**2,li)))
```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
3.py"
enter a list of integer = 3 4 5 6 7 8 3 2
[9, 16, 25, 36, 49, 64, 9, 4]
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
3.py"
enter a list of integer = 3 4 5 6
[9, 16, 25, 36]
PS E:\3rd sem\Python-Lab-Work>

```

4. Given two Python lists, write a program to iterate both lists simultaneously and display items from list1 in original order and items from list 2 in reverse order.

**Solution:**

```

from itertools import zip_longest
l1= [1,2,3,4,5]
l2=[1,2,3,4,5,6,7]
for a,b in zip_longest(l1,reversed(l2),fillvalue=0):
    print(a,b)

```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
4.py"
1 7
2 6
3 5
4 4
5 3
0 2
0 1
PS E:\3rd sem\Python-Lab-Work>

```

5. Write a program to count the number of occurrences of item 50 in the tuple tp1 = (50, 10, 60, 70, 50).

**Solution:**

```

tp1=(50,10,60,70,50)
print("tuple = ",tp1)
count=tp1.count(50)
print("number of occurrences = ", count)

```

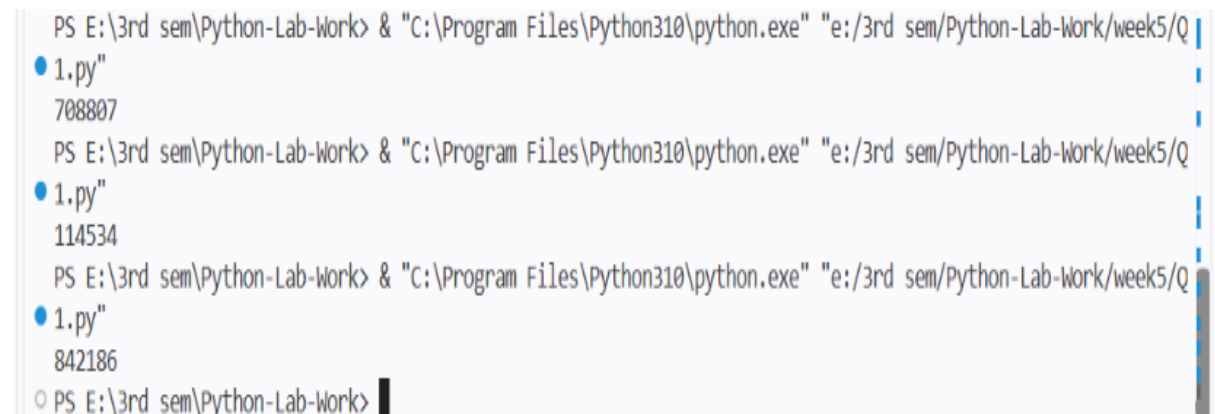
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
5.py"
tuple = (50, 10, 60, 70, 50)
number of occurrence = 2
PS E:\3rd sem\Python-Lab-Work> \
```

## Week #5

### 1. Write a program to generate a 6-digit random secure OTP.

#### Solution:

```
import secrets
otp= secrets.randbelow(900000)+100000
print(otp)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
708807
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
114534
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
842186
PS E:\3rd sem\Python-Lab-Work>
```

### 2. Write a program to pick a random character from a user-supplied string.

#### Solution:

```
import secrets
str= input("enter a string = ")
charac= secrets.choice(str)
print(charac)
```



```
enter a string = The world is beautiful
d
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
2.py"
enter a string = Hello
l
PS E:\3rd sem\Python-Lab-Work>
```

3. Write a program to generate a random password that meets the following conditions:
- a. Password length must be 10 characters long.
  - b. It must contain at least 2 uppercase letters, 1 digit, and 1 special symbol.

**Solution:**

```
import random
import string

uppercase = random.choices(string.ascii_uppercase, k=2)
digit = random.choice(string.digits)
special = random.choice("!@#$%^&*")

remaining = random.choices(
    string.ascii_letters + string.digits + "!@#$%^&*",
    k=6
)
password_list = uppercase + [digit] + [special] + remaining

random.shuffle(password_list)

password = ''.join(password_list)

print("Random Password:", password)
```



4. Given two lists of numbers, write a program to create a new list containing odd numbers from the first list and even numbers from the second list.

**Solution:**

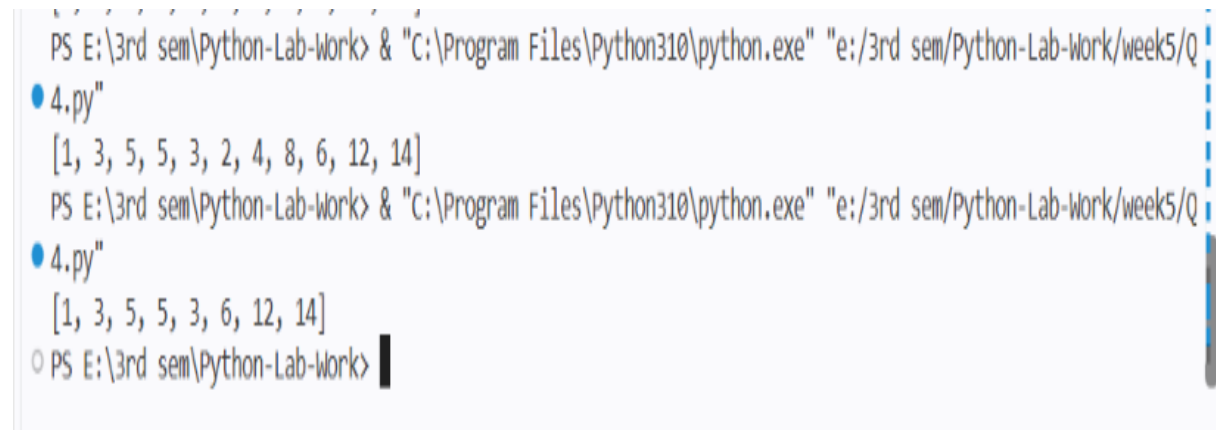
```
l1= [1,2,3,4,5,6,5,3]
l2= [2 ,3, 4,8, 7, 6, 5,12,14]
```

```

new_l=[]

for i in l1:
    if i%2!=0:
        new_l.append(i)
for i in l2:
    if i%2==0:
        new_l.append(i)
print(new_l)

```



```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
4.py"
[1, 3, 5, 5, 3, 2, 4, 8, 6, 12, 14]
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
4.py"
[1, 3, 5, 5, 3, 6, 12, 14]
PS E:\3rd sem\Python-Lab-Work>

```

5. Write a program to create a numpy array and return an array of odd rows and even columns from the numpy array.

### Solution:

```

import numpy as np
arr = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16],
    [17, 18, 19, 20]
])
result = arr[::2, 1::2]
print("Original Array:")
print(arr)

print("Odd rows and even columns:")
print(result)

```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
5.py"
Original Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
Odd rows and even columns:
[[ 2  4]
 [10 12]
 [18 20]]

```

**6. Write a program to create a numpy array and sort it as per the following cases:**

- a. Case 1: Sort the array by the second row.
- b. Case 2: Sort the array by the second column.

**Solution:**

```

import numpy as np

arr = np.array([
    [30, 20, 10],
    [3, 1, 2],
    [9, 7, 8]
])

print("Original Array:")
print(arr)

# a. Case 1: Sort the array by the second row.

index = np.argsort(arr[1])

result = arr[:, index]

print("Sorted by second row:")
print(result)

# b. Case 2: Sort the array by the second column.
index = np.argsort(arr[:, 1])

result = arr[index]

print("Sorted by second column:")

```

```
print(result)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
6.py"
• Original Array:
[[30 20 10]
 [ 3  1  2]
 [ 9  7  8]]
Sorted by second row:
[[20 10 30]
 [ 1  2  3]
 [ 7  8  9]]
Sorted by second column:
[[ 3  1  2]
 [ 9  7  8]
 [30 20 10]]
PS E:\3rd sem\Python-Lab-Work> █
```

## **Week #6**

- 1. Write a Python program that inputs two tuples and creates a third that contains all elements of the first followed by all elements of the second.**
- 2. Write a Python program to create a dictionary with names and phone numbers. Then ask the user for a name and print the corresponding phone number.**
- 3. Write a Python program to calculate the sum of squares of the first two digits and the last two digits of a 4-digit number, e.g., for 1233, calculate  $12^2 + 33^2$ .**
- 4. Write a program that inputs a main string and creates an encrypted string by embedding a short symbol-based string after each character. The program should also decrypt the string.**
- 5. Write a program to get roll numbers, names, and marks of students and store these details in a file called "Marks.data".**
- 6. Write a program to accept a string and display:**
  - a. Number of uppercase characters.**
  - b. Number of lowercase characters.**
  - c. Total number of alphabets.**
  - d. Number of digits.**
- 7. Create a C program that accepts an IPv4 address and subnet mask in dotted decimal format, and outputs:**
  - a. Network ID**
  - b. Broadcast Address**
  - c. First and Last Host Address**
  - d. CIDR notation**
  - e. Number of valid hosts**
  - f. IP Address Class (A/B/C/D/E)**

## **Week #7**

- 1. Write a report on different types of Linux shells (sh, csh, ksh, bash).**
- 2. Copy, move, and remove files using cp, mv, and rm commands.**
- 3. Create and delete directories using mkdir and rmdir.**
- 4. Change the current working directory using cd and display the present working directory using pwd.**
- 5. Consider two files that contain information about Employees and Departments in the following parameters: Employee (Name, EId, Salary, DID), Department (DID, DName, DLocation). Write a Python program to find the average salary of each department.**
- 6. Consider two files having some lines of statements. Write a Python program to swap content present at middle line of first file with the content of last line of the second file. (Note: First file contains odd numbers of lines of statement)**
- 7. Simulate a RIP-like behavior using a Python program. Accept a list of directly connected routers with hop counts and simulate RIP updates for 3 rounds. Show how the routing table evolves.**